Embedded Linux & System Software Interview Handbook

Chapter 5 - Linux Device Drivers (Part 2)

Topics: Linux Kernel Modules (LKM), Module Loading, printk(), dmesg and Module Life Cycle

5.5 What is a Linux Kernel Module (LKM)?

A Linux Kernel Module is a piece of code that can be loaded into or removed from the running kernel without rebooting the system. This enables developers to add device drivers and kernel functionality dynamically.

Advantages of LKMs

• Faster development and testing
• No kernel recompilation for every change
• Smaller base kernel
• Easy maintenance and upgrades

Module Life Cycle

Write Driver → Build Module (.ko) → insmod/modprobe → module_init() → Driver Registration → Device Available → rmmod → module_exit()

Important Macros

module_init(init_function)
module_exit(exit_function)
MODULE_LICENSE("GPL")
MODULE_AUTHOR("...")
MODULE_DESCRIPTION("...")

Common Commands

Command	Purpose
insmod driver.ko	Load module
rmmod driver	Unload module
modprobe driver	Load module with dependencies
lsmod	List loaded modules
modinfo driver.ko	Display module metadata
dmesg	Read kernel log buffer

printk() vs printf()

Kernel code cannot use printf(). Instead it uses printk(), which writes messages to the kernel log buffer. These messages can be viewed using dmesg or the system journal.

Kernel Log Levels

KERN_EMERG, KERN_ALERT, KERN_CRIT, KERN_ERR, KERN_WARNING, KERN_NOTICE, KERN_INFO, KERN_DEBUG

Minimal Module Skeleton

```
static int __init my_init(void){ printk(KERN_INFO "Driver loaded\n");
return 0; }
static void __exit my_exit(void){ printk(KERN_INFO "Driver unloaded\n");
}
module_init(my_init);
module_exit(my_exit);
```

Interview Discussion

Q: What is the difference between insmod and modprobe?
A: insmod loads only the specified module. modprobe also resolves and loads module dependencies using the module dependency database.

Real Debugging Workflow

1. Build the driver (.ko).
2. Load using modprobe or insmod.
3. Verify with lsmod.
4. Inspect logs using dmesg.
5. Test the device.
6. Unload with rmmod and verify cleanup.

Interview Questions

1. What is a Linux Kernel Module?
2. Why are LKMs preferred over rebuilding the kernel?
3. Explain module_init() and module_exit().
4. Difference between insmod and modprobe.
5. Why does kernel code use printk() instead of printf()?
6. How do you debug a driver that fails during loading?

Key Takeaways

Linux Kernel Modules provide a flexible mechanism for developing and deploying drivers. Understanding module loading, logging, and debugging is fundamental for embedded Linux interviews and day-to-day driver development.